

Pila de valores enteros

Una pila es una estructura de datos de tipo **LIFO** (*Last In, First Out*), crucial en programación para gestionar la memoria, realizar deshacer/rehacer en aplicaciones y manejar la recursión.

Implementa las clases indicadas a continuación usando técnicas de **programación orientada a objetos** en Python.

Recuerda que puedes añadir todos aquellos métodos auxiliares que creas necesarios.

1. Clase IntegerStack

Representa una pila de valores enteros.

top	4
	2
	6
	1
	7
	3



Almacena la pila como una **lista** donde el **top** de la pila esté en la posición 0:
[4, 2, 6, 1, 7, 3]

Implementa, *al menos*, los siguientes **métodos** (*tratando de hacer uso de métodos que vayas implementando*):

```
def __init__(self, *, max_size: int = 10):
```

- Crea los atributos: `items` y `max_size`.
- Ten en cuenta que `items` almacenará los elementos de la pila.

```
def __len__(self) -> str:
```

- Devuelve el número de elementos que hay en la pila `self`.

```
def is_empty(self) -> bool:
```

- Indica si la pila `self` está vacía.

```
def is_full(self) -> bool:
```

- Indica si la pila `self` está llena.

```
def push(self, item: int) -> bool:
```

- Añade `item` a la pila `self`.
- Si la pila está llena habrá que retornar `False`. En otro caso retornar `True`.

```
def pop(self) -> int | None:
```

- Extrae (y devuelve) el elemento que está en el *top* de la pila `self`.
- Si la pila está vacía habrá que retornar `None`.

```
def top(self) -> int | None:
```

- Devuelve el elemento que está en el *top* de la pila `self`.
- Si la pila está vacía habrá que retornar `None`.

```
def expand(self, factor: int = 2) -> None:
```

- Expande el tamaño máximo de la pila `self` en el factor indicado.
- Por ejemplo, si el tamaño máximo es 20 y el factor es 3, el tamaño máximo pasa a ser $20 \times 3 = 60$.

```
def dump_to_file(self, path: str) -> None:
```

- Vuelca la pila `self` a un fichero con ruta `path`.
- Cada elemento de la pila en una línea diferente.
- El primer elemento del fichero coincide con el elemento del *top* de la pila.

```
def load_from_file(cls, path: str) -> IntegerStack:
```

- Es un **método de clase**.
- Construye (y devuelve) una pila desde el fichero con ruta `path`.
- Cada item de la pila en una línea diferente.
- El primer elemento del fichero coincide con el elemento del *top* de la pila.
- Si la pila se llena al ir añadiendo elementos, habrá que expandir la pila mediante `expand()` con el valor por defecto de `factor`.

```
def __getitem__(self, index: int) -> int:
```

- Devuelve el elemento de la pila `self` que ocupa la posición (*índice*) `index`.

```
def __setitem__(self, index: int, item: int) -> None:
```

- Asigna el elemento `item` en la posición (*índice*) `index` de la pila `self`.

```
def __add__(self, other: IntegerStack) -> IntegerStack:
```

- Suma las pilas `self` y `other`.
- La segunda pila `other` va “encima” de la primera `self`.
- El tamaño máximo de la pila resultante es la suma de los tamaños máximos de cada pila.
- Devuelve la pila resultante.

```
def __iter__(self) -> IntegerStackIterator:
```

- Devuelve un objeto iterador de tipo `IntegerStackIterator`.

2. Clase `IntegerStackIterator`

Representa un iterador para la clase `IntegerStack`.

Implementa, *al menos*, los siguientes **métodos**:

```
def __init__(self, stack: IntegerStack) -> None:  
    ...
```

```
def __next__(self) -> int:  
    ...
```